

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I



BÁO CÁO TUẦN
HỌC PHẦN: THỰC TẬP CƠ SỞ
ĐỀ TÀI XÂY DỰNG HỆ THỐNG ĐIỂM DANH SINH VIÊN
THỜI GIAN THỰC DỰA TRÊN NHẬN DIỆN KHUÔN MẶT
VÀ QUẢN LÝ PHIÊN HỌC

Giảng viên hướng dẫn	: TS. Kim Ngọc Bách
Mã sinh viên	: B23DCCN579
Tên sinh viên	: Lê Minh Nam
Lớp hành chính	: D23CQCN05-B
Nhóm thực tập	: 11

Hà Nội – 2026

MỤC LỤC

XÂY DỰNG HỆ THỐNG	3
I. Mục tiêu	3
II. Nội dung nghiên cứu	3
2.1. Tối ưu hệ thống nhận diện khuôn mặt thời gian thực.....	3
2.1.1. Giảm tải xử lý bằng kỹ thuật skip frame	3
2.1.2. Tối ưu tính toán độ tương đồng cosine similarity	4
2.1.3. Cơ chế cooldown chống check-in lặp	5
2.1.4. Đồng bộ dữ liệu điểm danh định kỳ	5
2.2. Tối ưu hóa nhận diện khuôn mặt bằng DeepSORT Tracking	6
III. Kết luận	7

XÂY DỰNG HỆ THỐNG

I. Mục tiêu

Trong tuần này, mục tiêu chính của em là tối ưu hệ thống nhận diện khuôn mặt realtime nhằm nâng cao hiệu năng xử lý, độ ổn định và độ chính xác trong quá trình điểm danh sinh viên tự động.

Để đạt được mục tiêu trên, em tập trung nghiên cứu và triển khai các giải pháp tối ưu cho pipeline xử lý realtime như giảm tải xử lý bằng kỹ thuật skip frame; tối ưu tính toán cosine similarity bằng vector hóa dữ liệu embedding; bổ sung cơ chế cooldown nhằm hạn chế check-in lặp; đồng bộ dữ liệu điểm danh định kỳ giữa client và server.

Ngoài ra, hệ thống còn được tích hợp thuật toán DeepSORT Tracking nhằm theo dõi khuôn mặt giữa các frame liên tiếp, giúp giảm số lần thực hiện nhận diện embedding, tăng FPS xử lý realtime và nâng cao tính ổn định của quá trình nhận diện.

Thông qua các cải tiến trên, mục tiêu hướng tới là xây dựng hệ thống điểm danh tự động có khả năng hoạt động ổn định trong môi trường thực tế, xử lý tốt nhiều khuôn mặt đồng thời và hỗ trợ giảng viên theo dõi quá trình điểm danh một cách trực quan, hiệu quả hơn.

II. Nội dung nghiên cứu

2.1. Tối ưu hệ thống nhận diện khuôn mặt thời gian thực

Sau khi xây dựng chức năng điểm danh tự động bằng nhận diện khuôn mặt, tiến hành tối ưu module realtime nhằm nâng cao hiệu năng xử lý, độ ổn định và độ chính xác của hệ thống khi hoạt động thực tế.

2.1.1. Giảm tải xử lý bằng kỹ thuật skip frame

Trong quá trình stream video từ camera, hệ thống không thực hiện nhận diện trên toàn bộ frame mà chỉ xử lý sau mỗi N frame thông qua biến:

PROCESS_EVERY_N_FRAMES

```

1  if frame_count % PROCESS_EVERY_N_FRAMES == 0:
2      # Resize frame
3      resized_frame = cv2.resize(
4          frame,
5          (640, 360)
6      )
7
8      # update size cho YuNet
9      face_detector.setInputSize((640, 360))
10
11     # detect face
12     _, faces = face_detector.detect(resized_frame)

```

Giải pháp này giúp giảm số lượng phép tính cần thực hiện trong thời gian thực, từ đó:

- Giảm tải cho CPU/GPU.
- Tăng tốc độ xử lý.
- Hạn chế hiện tượng giật lag khi stream video realtime.

2.1.2. Tối ưu tính toán độ tương đồng cosine similarity

Embedding của toàn bộ sinh viên được chuyển thành ma trận NumPy và chuẩn hóa trước khi thực hiện nhận diện.

```

1  # Matrix embeddings
2  embedding_matrix = np.array([
3      item["embedding"]
4      for item in embeddings
5  ], dtype=np.float32)
6
7  matrix_norms = np.linalg.norm(
8      embedding_matrix,
9      axis=1,
10     keepdims=True
11 )
12
13 matrix_norms[matrix_norms == 0] = 1
14
15 embedding_matrix = embedding_matrix / matrix_norms

```

Khi có embedding mới từ khuôn mặt phát hiện được, hệ thống sử dụng phép toán vector hóa để tính toán cosine similarity.

```

1  sims = cosine_similarity_matrix(
2      embedding_matrix,
3      emb
4  )
5
6  best_idx = np.argmax(sims)
7
8  score = float(
9      sims[best_idx]
10 )

```

Việc vector hóa dữ liệu giúp:

- Giảm thời gian xử lý.
- Tăng tốc độ nhận diện.
- Tối ưu hiệu năng khi số lượng sinh viên lớn.

2.1.3. Cơ chế cooldown chống check-in lặp

Sau khi một sinh viên được điểm danh thành công, hệ thống sẽ áp dụng khoảng thời gian chờ trước khi cho phép check-in tiếp theo thông qua biến:

CHECKIN_COOLDOWN

```

1  if (
2      student_id not in last_check_in
3      or now_ts - last_check_in[student_id]
4      > CHECKIN_COOLDOWN
5  ):

```

Giải pháp này giúp:

- Tránh gửi nhiều request trùng lặp.
- Giảm tải cho backend.
- Đảm bảo dữ liệu điểm danh chính xác.

2.1.4. Đồng bộ dữ liệu điểm danh định kỳ

Hệ thống định kỳ đồng bộ trạng thái điểm danh từ backend để cập nhật dữ liệu mới nhất.

```

1  # REFRESH trạng thái điểm danh
2  if time.time() - last_sync > SYNC_INTERVAL:
3      checked_student_ids = get_checked_students(
4          class_id,
5          session_id,
6          token
7      )
8
9      last_sync = time.time()

```

Việc đồng bộ định kỳ giúp:

- Hạn chế sai lệch dữ liệu.
- Đồng bộ trạng thái giữa client và server.
- Hỗ trợ nhiều thiết bị theo dõi cùng lúc.

2.2. Tối ưu hóa nhận diện khuôn mặt bằng DeepSORT Tracking

Trong phiên bản ban đầu, hệ thống thực hiện nhận diện khuôn mặt trên mọi khung hình. Cách tiếp cận này làm tăng đáng kể chi phí tính toán do mô hình embedding phải chạy liên tục cho cùng một khuôn mặt qua nhiều frame liên tiếp.

Để cải thiện hiệu năng, nhóm đã tích hợp thuật toán DeepSORT nhằm theo dõi đối tượng khuôn mặt giữa các frame liên tiếp.

Quy trình hoạt động gồm các bước:

1. Sử dụng YuNet để phát hiện khuôn mặt định kỳ.
2. DeepSORT gán ID theo dõi (track ID) cho từng khuôn mặt.
3. Hệ thống chỉ thực hiện nhận diện embedding đối với track mới xuất hiện hoặc track cần xác minh lại định kỳ.
4. Sau khi nhận diện thành công, danh tính sinh viên được gán với track ID tương ứng.
5. Các frame tiếp theo sử dụng kết quả tracking thay vì nhận diện lại toàn bộ.

Việc kết hợp tracking giúp:

- Giảm số lần tính embedding.
- Tăng FPS của hệ thống realtime.
- Giảm hiện tượng nhấp nháy nhãn nhận diện.
- Cải thiện tính ổn định khi khuôn mặt di chuyển liên tục.

III. Kết luận

Sau quá trình nghiên cứu và triển khai, em đã tối ưu đáng kể hệ thống nhận diện khuôn mặt realtime cho bài toán điểm danh sinh viên tự động.

Hệ thống đã áp dụng thành công kỹ thuật skip frame nhằm giảm số lượng frame cần xử lý, từ đó giảm tải cho CPU/GPU và tăng tốc độ xử lý realtime. Đồng thời, việc vector hóa dữ liệu embedding bằng NumPy giúp tối ưu quá trình tính toán cosine similarity, cải thiện hiệu năng nhận diện khi số lượng sinh viên lớn.

Ngoài ra, cơ chế cooldown check-in và đồng bộ dữ liệu định kỳ đã giúp hạn chế request trùng lặp, giảm tải cho backend và đảm bảo tính nhất quán của dữ liệu điểm danh giữa client và server.

Bên cạnh các tối ưu trên, hệ thống đã được tích hợp thuật toán DeepSORT Tracking để theo dõi khuôn mặt giữa các frame liên tiếp. Giải pháp này giúp hạn chế việc nhận diện embedding lặp lại trên cùng một khuôn mặt, giảm chi phí tính toán, tăng FPS xử lý realtime và cải thiện độ ổn định của nhãn nhận diện khi đối tượng di chuyển.

Qua quá trình kiểm thử, hệ thống realtime hoạt động ổn định hơn, giảm hiện tượng giật lag và nâng cao khả năng nhận diện trong môi trường thực tế. Các cải tiến này tạo nền tảng quan trọng để tiếp tục tối ưu và mở rộng hệ thống điểm danh sử dụng AI trong thời gian tới.